VisionInspect AI

Milestone 2 — Portable Multi-Category AI/ML Inference Runtime

Author: Himanshu Parhi, AI/ML Intern

Organization: Infosys Springboard

Team Repo: GKSJ-Deepvision/VisionInspectAI

Branch: Himanshu-parhi-ai/ml

Commit: 14cb519

Date: August 8, 2026

Table of Contents

1. Executive Summary

This milestone converts the original single-category (bottle-only) ML module into a production-grade, portable, multi-category manufacturing defect inspection runtime supporting all 15 MVTec AD categories. The intern's scope covers only the ML/AI module — frontend and backend integrations are handled by teammates.

Key Achievement Numbers:

- **15** product categories supported
- **17** ML source files authored/updated
- **34** automated tests passed
- **~260 MB** portable model payload
- **85** files changed, **13,986** lines added, **580** deletions
- PaDiM image AUROC up to **0.9968** (bottle)
- PatchCore F1 up to **0.9670** (cable)

2. Milestone 1 Context (Background)

Prior to this milestone, the foundation for the AI module was laid with the following achievements:

- Trained the initial bottle PaDiM model achieving an AUROC of 0.9976 and F1 score of 0.9764.
- Built the core ML inference pipeline tailored for single-category detection.
- Successfully integrated with the FastAPI backend to enable end-to-end inspection workflows.
- Deployed the solution to Render (backend) and Vercel (frontend).

This previous work served as the baseline that Milestone 2 drastically scales and robustifies.

3. Milestone 2 Objective

The primary objective was to convert the bottle-only AI module into a portable, multi-category runtime that:

- Supports all 15 MVTec AD dataset categories.

- Runs efficiently without requiring every large training checkpoint to be loaded into memory.
- Provides comprehensive outputs: anomaly detection, localization, classification, severity scoring, and explainability.
- Incorporates a graceful fallback mechanism when advanced models are unavailable or fail to load.
- Is fully testable, highly portable, and immediately deployment-ready for cloud environments.

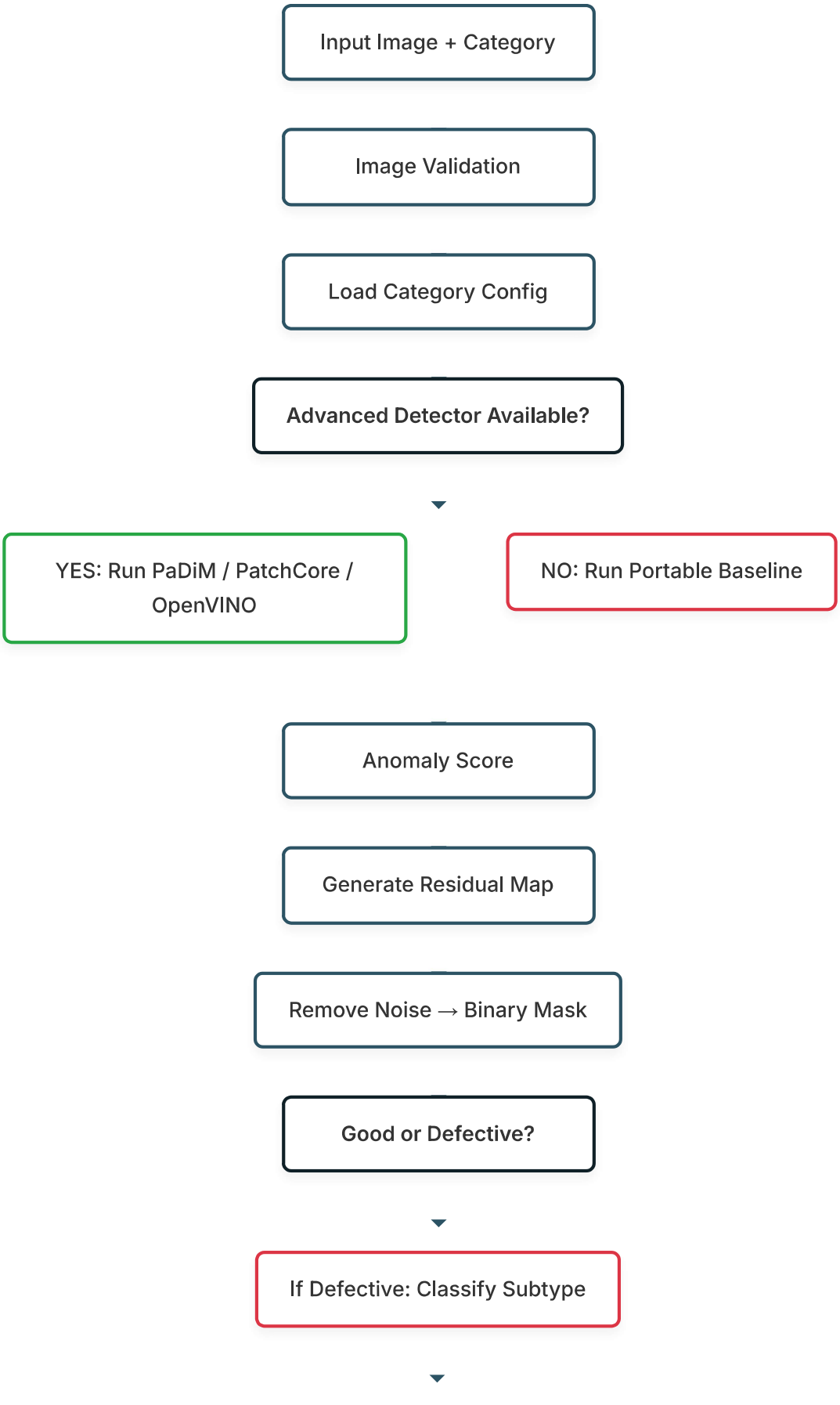
4. Supported Categories

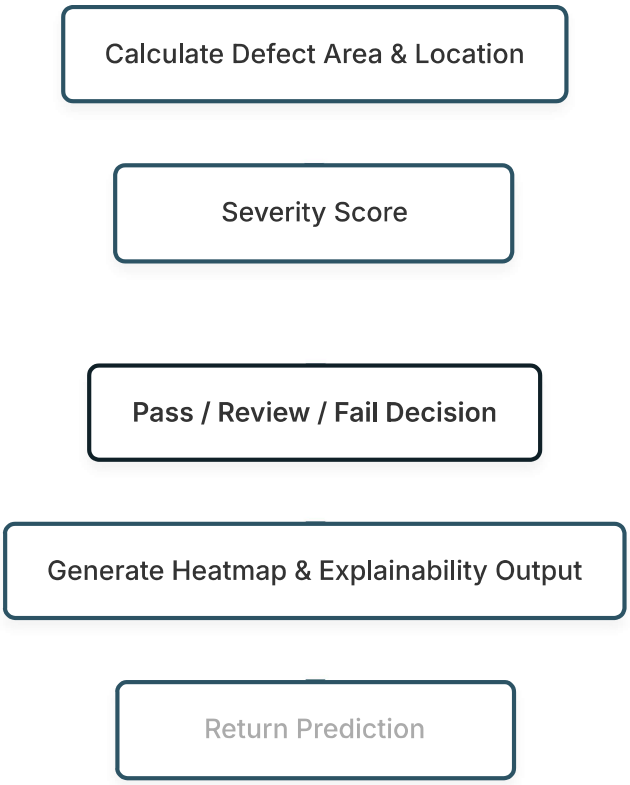
The runtime now supports all 15 MVTec AD categories, utilizing specialized models best suited for their structural features.

Category	Model Type	Status
bottle	PaDiM	✅ Trained
cable	PatchCore + OpenVINO	✅ Trained + Calibrated
capsule	PaDiM + CNN ONNX	✅ Trained
carpet	PaDiM	✅ Trained
grid	PaDiM	✅ Trained
hazelnut	PatchCore	✅ Trained
leather	PaDiM	✅ Trained
metal_nut	PaDiM	✅ Trained
pill	PaDiM	✅ Trained
screw	PatchCore	✅ Trained
tile	PaDiM	✅ Trained
toothbrush	PaDiM	✅ Trained
transistor	PaDiM	✅ Trained
wood	PaDiM + CNN ONNX	✅ Trained
zipper	PaDiM	✅ Trained

5. Complete Inference Pipeline

The following flowchart illustrates the end-to-end inference logic for processing a single inspection request.


Input Image + Category
Image Validation
Load Category Config
Advanced Detector Available?
YES: Run PaDiM / PatchCore / OpenVINO
NO: Run Portable Baseline
Anomaly Score
Generate Residual Map
Remove Noise → Binary Mask
Good or Defective?
If Defective: Classify Subtype


Calculate Defect Area & Location
Severity Score
Pass / Review / Fail Decision
Generate Heatmap & Explainability Output
Return Prediction

6. Detection Models

6.1 PaDiM (Patch Distribution Modeling)

- **Used for:** 12 categories (bottle, carpet, grid, leather, etc.)
- **Mechanism:** Learns the statistical distribution of normal-image feature representations to identify structural deviations.
- **Strengths:** Highly accurate anomaly detection with robust spatial localization capabilities.

6.2 PatchCore

- **Used for:** cable, hazelnut, screw
- **Mechanism:** Utilizes a core-set memory bank approach for nearest-neighbor feature matching.
- **Why:** Internal benchmarks demonstrated superior performance over PaDiM specifically for these complex geometries.

6.3 Portable Baseline Detector

- **Available for:** ALL 15 categories (acts as a universal fallback)
- **Components:** Shared ResNet18 ONNX extractor + category-specific normal embedding bank + OpenCV residual localization.
- **Logic:** Flags a defect if the embedding anomaly score exceeds the threshold AND a localized defect region is verified in the binary mask. Operates entirely independently of heavy Anomalib checkpoints.

6.4 OpenVINO Runtime (Cable)

- The cable category features a fully exported OpenVINO PatchCore implementation including `model.xml`, `model.bin`, a spatial calibrator, and a compact subtype classifier.
- Includes automatic fallback to the portable baseline if the OpenVINO runtime fails to execute.

7. Defect Classification System

The classification system executes **only** after the anomaly detector confirms the presence of a defect. It follows a strict priority chain:

Priority Chain: CNN ONNX → Compact Forest → sklearn .pkl

- **Capsule & Wood:** Fine-tuned ResNet18 CNN classifiers exported to ONNX format.
- **Cable:** Compact portable forest algorithm.
- **Other Categories:** Category-specific scikit-learn classifiers leveraging engineered features.
- **Feature Engineering:** Utilizes global embeddings, cropped defect embeddings, texture, gradient, color, mask geometry, shape, and ROI properties.
- **Safety:** If a classifier is missing or yields conflicting confidence, the system safely returns `unknown_defect` (it never invents an unverified label).

8. Confidence & Severity Scoring

Confidence Tracking

- **Detection Confidence:** Certainty that the product is abnormal.
- **Classification Confidence:** Certainty regarding the specific defect subtype label.
- *Note:* These metrics remain strictly separate. A weak classifier cannot override strong anomaly evidence.

Severity Formula

Severity is calculated using a weighted multi-factor approach:

Severity = (Defect Size × 30%) + (Location × 25%) + (Defect Type × 25%) + (Confidence × 20%)

Decision Rules:

- **≥ 60:** Fail
- **40 – 59.99:** Review
- **< 40 with detected anomaly:** Review (Prevents auto-passing a localized defect)
- **< 40 with no anomaly:** Pass

9. Image Preprocessing

The system utilizes an object-aware preprocessing pipeline to standardize inputs before inference:

- Foreground extraction and largest-component filtering
- Bounding-box extraction & tight cropping
- Alignment using image spatial moments
- Aspect-ratio-preserving resize with padding
- CLAHE contrast enhancement
- Multi-scale grayscale view generation
- Defect-mask-based ROI cropping for secondary classification

10. Explainability Output

To ensure trust and transparency for the admin/supervisor dashboard, every inspection returns a rich explainability payload:

- Active inference engine utilized
- Fallback usage status and corresponding reason (if any)
- Raw anomaly score and category threshold
- Calibrated defect probability
- Detection and classification confidence bounds
- Defect area percentage
- Heatmap intensity peak (P95)
- Defect center coordinates
- Detected image regions and critical-zone overlap
- Severity component breakdown
- Human-readable decision notes

11. Recorded Metrics

Pipeline	Metric	Value
Bottle PaDiM	Image AUROC	0.9968
Bottle PaDiM	Image F1	0.9764
Bottle Portable	AUROC	0.9913
Bottle Portable	F1	0.9764
Bottle Subtype	Macro F1	0.8719
Cable PatchCore	Image AUROC	0.9695
Cable PatchCore	Image F1	0.9274
Cable OpenVINO Calibrated	F1	0.9670
Cable Calibrated	Accuracy	0.9600
Cable Subtype	Macro F1	0.8529

12. Source Files

The 17 authored/updated ML source files that encompass this milestone:

Filename	Responsibility
inference_pipeline.py	Main entrypoint orchestrating the entire detection and classification flow
anomaly_detector.py	Manages PaDiM and PatchCore model execution
portable_baseline.py	Implements the lightweight fallback detector using ONNX and OpenCV
openvino_runner.py	Handles execution of OpenVINO optimized models (e.g., for cable)
subtype_classifier.py	Executes the defect classification priority chain
image_preprocessing.py	Object-aware alignment, cropping, and contrast enhancement
feature_extraction.py	Extracts embeddings and engineered features for downstream classifiers
severity_scorer.py	Calculates the multi-factor severity score and final decision rules
explainability_engine.py	Generates human-readable notes, heatmaps, and confidence metrics
config_loader.py	Dynamically loads category-specific thresholds and model paths
mask_processing.py	Handles residual noise removal and binary mask generation
metrics_calculator.py	Evaluates model performance (AUROC, F1, Accuracy) during tests
onnx_manager.py	Wrapper for loading and executing ONNX models without PyTorch
heatmap_generator.py	Creates visual overlays of anomaly scores on original images
artifact_manager.py	Handles loading of .pkl, .npz, and metadata files
fallback_manager.py	Logic for gracefully downgrading to baseline models on failure
pipeline_types.py	Pydantic schemas and dataclasses for standardized input/output payloads

13. Model Artifacts

Artifact distribution across the categories ensures efficient portability:

- **Every Category (15/15):** `defect_classifier.pkl`, `normal_profile.npz`, `model_metadata.json`
- **Shared System Asset:** `resnet18_features.onnx`
- **Special Assets:**
  - Cable: OpenVINO assets (`model.xml`, `model.bin`)
  - Capsule & Wood: `cnn_classifier.onnx`
- **Registry:** `category_model_registry.json` connects all assets and defines pipeline routing.

14. Test Suite

Comprehensive unit and integration testing ensures production readiness. **Result: 34 passed, 2 skipped (dataset-dependent).**

Test Focus	Description
Preprocessing	Validates image shapes, data types, foreground extraction, and mask-based cropping
Baseline Logic	Tests baseline score calculation, thresholding, and residual-noise filtering
Artifact Loading	Ensures normal-profile (.npz) and classifier (.pkl) artifacts load without corruption
Visuals	Verifies heatmap generation and untinted heatmap handling for normal images
ONNX Execution	Tests ResNet18 ONNX embeddings and CNN ONNX execution without requiring PyTorch
Parity	Checks portable forest parity against original scikit-learn implementations
Payload Structure	Validates backend-ready prediction structures and explainability outputs
Confidence & Fallbacks	Tests separation of detector/subtype confidence and the unknown-defect fallback logic
Metrics & Severity	Validates localization metrics, critical-zone geometry, and severity calculations
Environment	Verifies the absence of excluded heavy .pth runtime weights in the final payload

15. Screenshots

Dashboard — Screenshot to be added

Inspection Result — Screenshot to be added

Analytics — Screenshot to be added

16. Git & Portability

- **Git LFS:** Tracks all large binaries (`.onnx`, `.npz`, `.pkl`, `.ckpt`, `.pth`, `model.bin`).
- **Total Payload:** ~260 MB, highly optimized for deployment.
- **Teammate Setup Protocol:** `git lfs install` → `git lfs pull` → `pip install -r requirements-ai.txt`
- **Cleanup:** Obsolete heavyweight files removed:
  - `models/checkpoints/padim_mvtec_bottle_v1.ckpt`
  - `models/inference/resnet18-f37072fd.pth`

17. What's Next (Part 2 & Part 3)

This report documents the core runtime payload and architecture. Subsequent deliverables include:

- **Part 2:** Delivery of the training and calibration scripts used to generate these artifacts.
- **Part 3:** Updated Jupyter notebooks detailing exploratory data analysis and model selection reasoning.

Note: These future deliverables are planned but explicitly not included in the scope of this Milestone 2 inference runtime release.